

Lecture: 0110

Counters

CSA222: Modern Computer Architecture

SECTION 02

2. Counters

The Beating Clock of Digital Systems

Flip-flops configured to count clock pulses, serving as the essential foundation for timing, addressing, and control flow.

CORE APPLICATIONS

**Timers & Clock Dividers**

Measuring precise time intervals and scaling frequencies.

**Address Generation**

Stepping through memory locations sequentially.

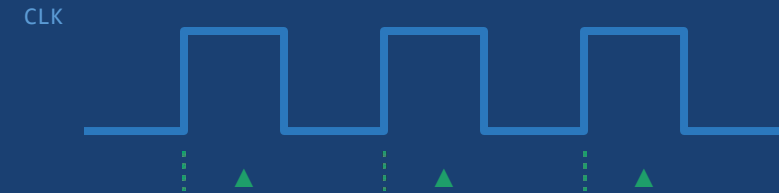
**The Program Counter (PC)**

The engine driving instruction fetch and CPU execution flow.

VISUALIZING CLOCK & COUNT

How a Counter Advances

Each rising edge of the clock trigger increments the register state.

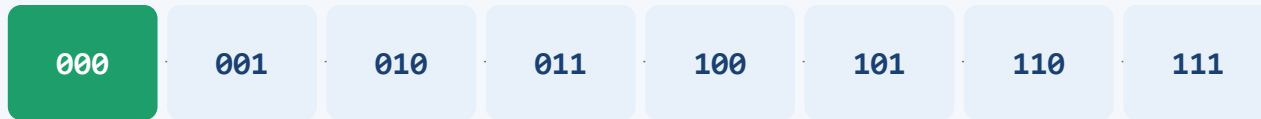


STATE TRANSITION (N-BIT OVERFLOW)

What Is a Counter?

A register whose value advances by 1 on every clock pulse — it counts.

A 3-BIT COUNTER CYCLES THROUGH 0 → 7 → 0...



↻ wraps back to 000

Each clock pulse = +1.

After the max value (111 = 7), it rolls over to 000 — exactly like an odometer, or 8-bit overflow from L4.

An n -bit counter counts 0 to 2^n-1 , then wraps. It's a register + an incrementer + feedback.

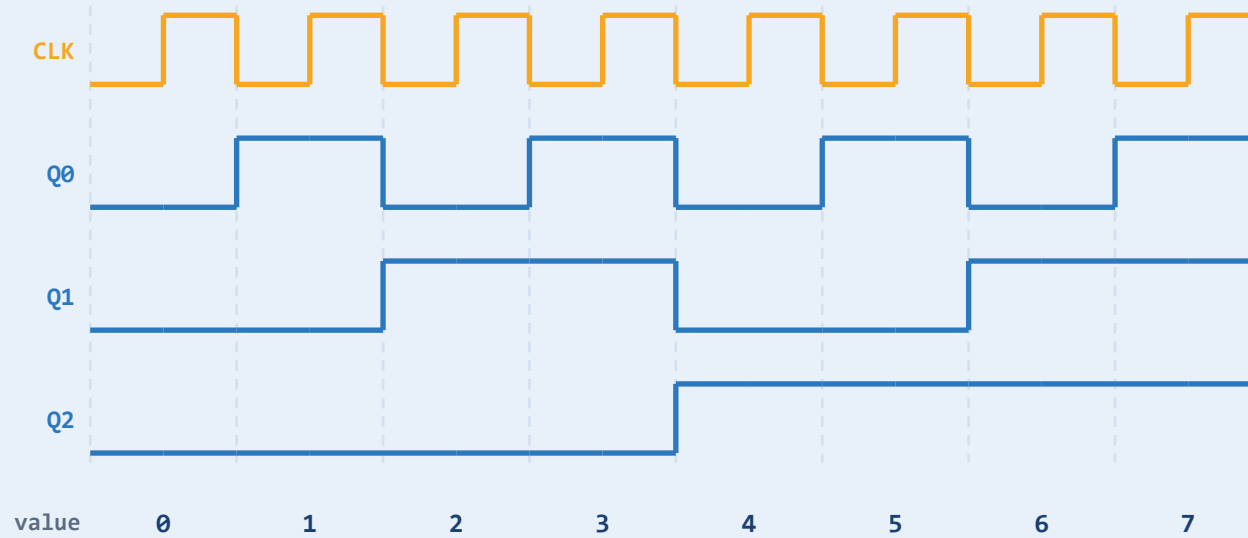
WHY COUNTERS MATTER

The Program Counter (PC) is a counter — it holds the address of the next instruction and increments each cycle.

Also: timers, clock dividers, memory-refresh logic, event counters — all built on this.

Reading the Count as Waveforms

Look at the bits over time and the pattern that makes counters buildable jumps out.



Q0 changes every clock. Q1 changes every second clock. Q2 changes every fourth.

THE RULE BEHIND IT

A bit flips exactly when every bit below it is 1.

Q0 flips always. Q1 flips when Q0 = 1. Q2 flips when Q1·Q0 = 1.

IN T FLIP-FLOP TERMS

$T0 = 1$

$T1 = Q0$

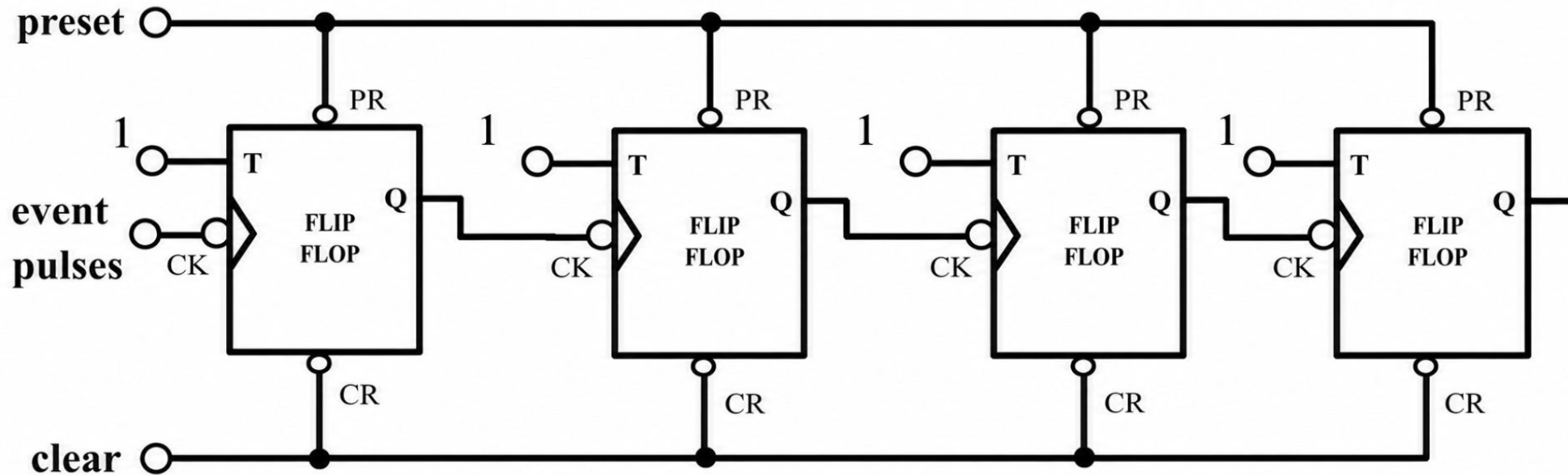
$T2 = Q0 \cdot Q1$

THAT IS THE WHOLE DESIGN

Nothing else is needed for a plain binary up-counter. Two different circuits implement it — and choosing between them is the next four slides.

Ripple (Asynchronous) Counter

Each flip-flop's output clocks the next — simple, but the carry ripples.



Only FF0 sees the real clock; each next FF is clocked by the previous output — the change 'ripples' along.

⚠ The ripple delay means the count is briefly **WRONG** while it propagates — a problem at high speed.

Why Ripple Counters Break at Speed

Follow the count from 7 to 8 one gate delay at a time, and watch three wrong numbers appear on the pins.

TIME AFTER EDGE	Q3	Q2	Q1	Q0	VALUE ON PINS
before	0	1	1	1	7 — correct
1 × t _{pd}	0	1	1	0	6 — wrong
2 × t _{pd}	0	1	0	0	4 — wrong
3 × t _{pd}	0	0	0	0	0 — wrong
4 × t _{pd}	1	0	0	0	8 — correct at last

Anything reading those pins during the ripple — a comparator, a decoder, a memory address bus — sees 6, then 4, then 0.

THE SPEED LIMIT

The count is only valid after ALL n stages have settled:

$$t_{\text{settle}} = n \times t_{\text{pd}}$$

$$f_{\text{max}} = 1 / (n \times t_{\text{pd}})$$

PUT NUMBERS ON IT

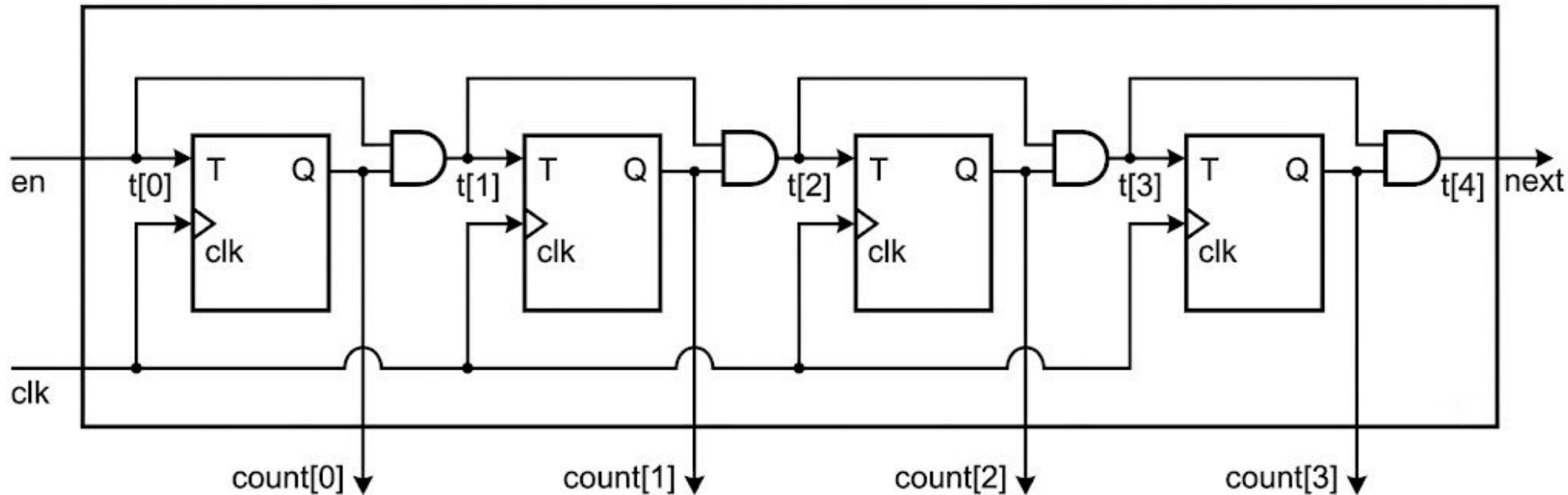
At $t_{\text{pd}} = 10 \text{ ns}$, a 4-bit ripple counter tops out near 25 MHz — and a 16-bit one near 6 MHz. A modern CPU clock is 3 GHz.

THE WAY OUT

Give every flip-flop the same clock, and compute in advance which ones should flip.

Synchronous Counter

All flip-flops share ONE clock — they all update at the exact same instant.



How It Works

Combinational logic (the & gates) computes each FF's next bit; the shared clock updates them all together.

Key Advantage

✓ No ripple delay — the whole count is correct the instant after the edge. This is what real designs use.

Ripple vs Synchronous

Same count sequence, very different timing behaviour.

	RIPPLE (async)	SYNCHRONOUS
Clocking	Each FF clocks the next	All FFs share one clock
Speed	Slow — carry ripples through	Fast — all update at once
Glitches	Yes — transient wrong counts	No — clean output
Complexity	Simple, fewer gates	Needs next-state logic
Real use	Toy / low-speed dividers	Everything performance-critical

Trade-off in one line: ripple is cheaper to build; synchronous is the only one safe at real clock speeds.

Mod-N & Divide-by Counters

Reset early to count any range — or use a counter to slow a clock down.

MOD-N: STOP EARLY

A 3-bit counter naturally counts mod-8 (0–7). To count mod-6 (0–5), detect the value 6 and reset to 0.



↻ reset at 6

Used for: clock dividers, digital clocks (mod-60 for seconds!), any custom cycle length.

DIVIDE-BY: SLOW THE CLOCK

Each flip-flop in a counter toggles at HALF the rate of the one before it.

So bit 0 = $\text{clock} \div 2$, bit 1 = $\text{clock} \div 4$, bit 2 = $\text{clock} \div 8$...

A counter is a frequency divider for free. This is how a 32 kHz crystal becomes a 1 Hz tick in a watch.

Counters in the Wild

Once you see them, they're everywhere.



Program Counter

Holds the address of the next instruction; increments each cycle, or jumps on a branch. The heart of instruction flow.



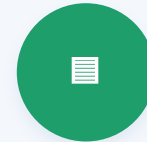
Timers & delays

Count clock ticks to measure time — every microcontroller timer, watchdog, and PWM generator is a counter.



Frequency division

Turn a fast crystal oscillator into slower usable clocks — watches, UART baud rates, display refresh.



Memory addressing

Sequentially step through addresses in DMA transfers, refresh cycles, and scan-out logic.



**Thanks
for
watching!**